The Journal of Open Source Software

zipstream-ai: A Python Package for Streaming and Querying Zipped Datasets with LLMs

Pranav Motarwar [1¶], **Rudra Patil**[1], **and Ashutosh Lembhe**[2]

1 New York University **2** University of Limerick ¶ Corresponding author

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 14 December 2025
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Summary

Compressed datasets provided as `.zip` or `.tar.gz` archives are becoming increasingly popular in exploratory data analysis workflows, mainly across the domains of academic research which involves large and complex datasets. These archives usually contain structured files, such as CSV, JSON, or TXT, but these data files also need pre-processing, which usually involves the user following procedures like file extraction, inspection, and before the data could be actually used in the research package. Currently, without human conversion or summarization, these files cannot be directly processed by contemporary AI technologies that use Large Language Models (LLMs).

The **zipstream-ai** is LLM-integrated Python package that allows users to stream, parse, and query structured files inside compressed archives without extraction. This package is valuable for users who frequently work with large and complex datasets which are often wrapped in a zip format and want to minimize latency in exploration or integrate them into AI pipelines.

zipstream-ai offers both a Python API and a command-line interface (CLI) for programmatic use. It auto-detects file formats, converts them into structured Python objects, and allows users to query the contents using natural language via Google's Gemini models.

Statement of Need

Compressed Datasets are the norm when it comes to modern research. However, conventional Python libraries like tarfile and zipfile lack essential usability aspects and only offer low-level archive access. They don't enable high-level interfaces like natural language searches, interact with tools like pandas, or automatically identify file kinds. Before any valuable insight can be discovered, users must manually unpack archives, examine files, and create boilerplate parsing templates. Such overhead work can limit the adoption of LLM-based technology in real-world processes.

In LLM-powered systems, where agents are supposed to interact with data seamlessly, these constraints are troublesome. A large percentage of real-world datasets are still inaccessible to LLM workflows in the absence of a smooth method for retrieving structured material from archives.

`zipstream-ai` addresses this gap by allowing users to directly stream and parse common file from within compressed archives and immediately interact with them using large language models. This enables AI-native exploration of zipped datasets without manual overhead.

The package supports both programmatic and CLI workflows. It's ideal for users who want to:

- interact with structured data inside compressed files without manual extraction,
- explore ZIP archives directly within Jupyter like environments,
- integrate compressed datasets into Retrieval-Augmented Generation (RAG) pipelines,
- perform natural language querying of archived data using LLMs.

Methodology

The architecture of `zipstream-ai` is built around three primary Python modules. `ZipStreamReader` allows users to directly stream and list contents from compressed `.zip` and `.tar.gz` archives without requiring extraction. This minimizes disk usage and accelerates the workflow, particularly for large datasets.

Secondly, `FileParser` automatically detects file formats such as `.csv`, `.json`, `.txt`, and `.md`, and parses them into structured Python-native objects, usually using `pandas`. It also detects common delimiters like commas, semicolons, or tabs, for flexible parsing.

The third component, `ask()`, enables natural language interaction with the parsed data by leveraging Google's Gemini models. This function constructs prompts from representative data and queries the LLM to generate insights or perform analytical reasoning.

To support diverse usage, the package includes `.env` file support for secure API key management via `python-dotenv`, and a command-line interface (CLI) built with `typer` that supports interactive browsing and querying. The architecture is streaming-safe and is designed to plug into both Jupyter-based workflows and backend pipelines. Planned extensions include support for OCR and integration with open-source LLMs for offline use.

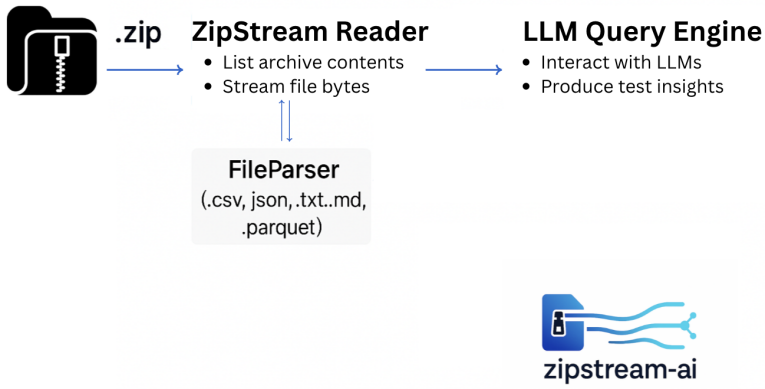

Figure 1: Architecture of *zipstream-ai*, showing how compressed archives are streamed, parsed, and queried using LLMs through its core modules.

Related Work

While Python provides built-in modules like zipfile and tarfile for archive access, they operate at a low level and do not integrate with data parsing or natural language interfaces. Libraries in pandas support reading from compressed paths like .zip archives (Smith, 2021) but only if the internal file name is known. They also do not support archive exploration or file listing. Other tools, such as Hugging Face Datasets (Sant et al., 2025), allow loading datasets from compressed formats but require predefined dataset configurations. Recent research has explored leveraging large language models (LLMs) for structured data interaction, including natural language interfaces for visualization generation, analytical reasoning, and API-driven code synthesis (Li et al., 2024; Narechania et al., 2020; Pochinkov et al., 2024). These approaches typically assume that data has already been extracted, cleaned, and loaded into tabular formats, skipping the upstream challenges of accessing and parsing compressed datasets unaddressed. Other systems have investigated LLM-assisted data preparation and retrieval-augmented generation (RAG) pipelines (Lewis et al., 2020), but they similarly rely on preprocessed inputs. zipstream-ai is the first open-source tool that combines archive streaming, format detection,

data parsing, and LLM-based natural language querying into a single lightweight interface usable via both Python and CLI.

Implementation

The package is implemented in Python 3.8+ and relies on several widely adopted libraries like `pandas` for data manipulation, `gemini` for LLM access, `typer` for building the command-line interface, and `python-dotenv` for managing environment variables securely. The LLM integration is handled via Google Gemini with extensive testing across both Gemini's 2.0 and 3.0 models to ensure compatibility.

The CLI is built using `typer`, allowing for clean argument parsing and quick extensibility. For development and testing, `pytest` is used, and the package supports editable installation to facilitate rapid iteration. The entire project is open-source under the MIT license, available on GitHub, and published on PyPI for ease of installation.

```python
from zipstream_ai import ZipStreamReader, FileParser
from zipstream_ai.llm_query_engine import ask

reader = ZipStreamReader("datasets/sample_data.zip")

# Listing files inside the archive
files = reader.list_files()
print(files)

# Parse a CSV file directly from the archive
parser = FileParser(reader)
df = parser.load("sales_data.csv")

# Query the data via an LLM
response = ask(df, "Which product category has the highest total sales?")
print(response)
```

Acknowledgements

This project was developed by Pranav Motarwar, Rudra Patil and Ashutosh Lembhe as part of their applied machine learning research work. The tool reflects an independent effort to streamline LLM-based interaction with compressed datasets.

References

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., & others. (2020). Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in Neural Information Processing Systems*, *33*, 9459–9474.

Li, Y., Tan, Z., & Xiao, W. (2024). LLM for uniform information extraction using multi-task learning optimization. *Asia-Pacific Web (APWeb) and Web-Age Information Management (WAIM) Joint International Conference on Web and Big Data*, 17–29.

Narechania, A., Srinivasan, A., & Stasko, J. (2020). NL4DV: A toolkit for generating analytic specifications for data visualization from natural language queries. *IEEE Transactions on Visualization and Computer Graphics*, *27*(2), 369–379.

Pochinkov, N., Benoit, A., Agarwal, L., Majid, Z. A., & Ter-Minassian, L. (2024). Extracting paragraphs from llm token activations. *arXiv Preprint arXiv:2409.06328*.

DRAFT

Sant, G., Jiang, Z., Escolano, C., Moryossef, A., Müller, M., Sennrich, R., & Ebling, S. (2025). MultimodalHugs: Enabling sign language processing in hugging face. *arXiv Preprint arXiv:2509.09729*.

Smith, D. S. (2021). RawArray: A simple, fast, and extensible archival format for numeric data. *arXiv Preprint arXiv:2112.01273*.

DRAFT